

IP Address Extraction Function

User

This is a Enriched Context Generation scenario.

The following content was found in the files provided by the user.

**** IpAddress.cpp full content ****

```
#include <string>
#include <stdexcept>
#include "IpAddress.hpp"
using namespace std;

// This entire constructor has been AI-generated.
IpAddress::IpAddress(std::string form) {
    /* ----- Allocate storage ----- */
    this->values = new int[4];
    this->port = -1;    // "no port" sentinel

    const char* s = form.c_str();    // raw C-style string
    size_t len = form.size();
    size_t i = 0;    // current index in the string

    /* ----- Parse the four octets ----- */
    for (int part = 0; part < 4; ++part) {
        if (i >= len)
            throw std::invalid_argument("IP address too short");

        int value = 0;
        int digits = 0;
        bool leadingZero = false;

        /* read the numeric characters of this octet */
        while (i < len && s[i] >= '0' && s[i] <= '9') {
            if (digits == 0) {    // first digit
                leadingZero = (s[i] == '0');
            } else {
                /* once we have more than one digit, a leading zero is forbidden */
                if (leadingZero)
                    throw std::invalid_argument("Octet has leading zeros");
            }

            value = value * 10 + (s[i] - '0');
            if (value > 255)    // range check
                throw std::invalid_argument("Octet out of range 0-255");

            ++i;
        }

        this->values[part] = value;
    }
}
```

```

        ++digits;
        ++i;
    }

    if (digits == 0)
        throw std::invalid_argument("Missing octet value");

    /* after the digits, we expect '.' or ':' (or end of string) */
    if (part &lt; 3) { // not the last part → must be '.'
        if (i &gt;= len || s[i] != '.')
            throw std::invalid_argument("Expected '.' between octets");
        ++i; // skip the dot
    } else { // last part
        /* we may have ':' for a port or end-of-string */
        if (i &lt; len && s[i] == ':') {
            /* keep colon, parse port later */
        } else if (i != len) {
            throw std::invalid_argument("Unexpected character after IP address");
        }
    }

    this->values[part] = value;
}

/* ----- Parse optional port ----- */
if (i &lt; len && s[i] == ':') { // colon found → parse port
    ++i; // skip ':'

    int pvalue = 0;
    int pdigits = 0;
    bool pLeadingZero = false;

    while (i &lt; len && s[i] &gt;= '0' && s[i] &lt;= '9') {
        if (pdigits == 0) { // first digit of the port
            pLeadingZero = (s[i] == '0');
        } else {
            if (pLeadingZero)
                throw std::invalid_argument("Port has leading zeros");
        }

        pvalue = pvalue * 10 + (s[i] - '0');
        if (pvalue &gt; 65535) // range check
            throw std::invalid_argument("Port out of range 0-65535");

        ++pdigits;
        ++i;
    }

    if (pdigits == 0)
        throw std::invalid_argument("Missing port number");

    /* after the digits there must be nothing else */
    if (i != len)
        throw std::invalid_argument("Unexpected characters after port");
}

```

```

        this->port = pvalue;
    } else if (i < len) { // stray character
        throw std::invalid_argument("Invalid trailing data");
    }
}

```

```

}
IpAddress::~IpAddress() {
    delete[] this->values;
}
string IpAddress::ToString() {
    char toReturn[32];
    char temp[32];
    for(int i = 0; i < 4; i++) {
        if(i > 0) {
            sprintf(temp, "%s.", toReturn);
        } else {
            temp[0] = '\0';
        }
        sprintf(toReturn, "%s%d", temp, this->values[i]);
    }
    if(this->IsPortDefined()) {
        sprintf(temp, "%s:%d", toReturn, this->GetPort());
        sprintf(toReturn, "%s", temp);
    }
    return string(toReturn);
}
int IpAddress::GetPort() {
    return this->port & 0xFFFF;
}
bool IpAddress::IsPortDefined() {
    return this->port >= 0 && this->port <= 0xFFFF;
}

```

**** end of IpAddress.cpp ****

**** IpAddress.hpp full content ****

```

#pragma once
#include <string>
using namespace std;

class IpAddress {
public:
    IpAddress(string form);

```

```

~IpAddress();
explicit operator int() {
int toReturn = 0;
for(int i = 0; i < 4; i++) {
toReturn <=< 8;
toReturn += 0xFF & (this->values + i);
}
return toReturn;
}
explicit operator string() {
return this->ToString();
}
string ToString();
int GetPort();
bool IsPortDefined();
protected:
int values;
int port;
};

```

**** end of IpAddress.hpp ****

**** IpDetector.cpp full content ****

```

#include "IpDetector.hpp"
#include <string>
using namespace std;

// I don't expect this do do anything
// This format was a product of CLion suggestions, not AI.
IpDetector::IpDetector() = default;
// Get the first potential IP address
// This does not guarantee it is valid, but does guarantee it only consists of characters
// That the IpAddress class should parse.
string IpDetector::GetPotentialIpString(string& input) {
// Automatically defined to empty string
string toReturn;
// Iterate through the string until we reach an invalid character.
int i = 0;
// This should not throw an error assuming the string is terminated with any reasonable character.
for (; this->IsValidCharacter(input[i]); i++) {
toReturn += input[i];
}
}

```

```

input = input.substr(i);
return toReturn;
}
bool IpDetector::extractIPv4(const string& str, unsigned long& outAddress, int& outPort) {
return false;
}

bool IpDetector::IsValidCharacter(char c) {
return (c >= '0' && c <= '9') || (c == '.') || (c == ':');
}

```

**** end of IpDetector.cpp ****

**** IpDetector.hpp full content ****

```

#pragma once
#include <string>
using namespace std;

class IpDetector {
public:
// I don't expect this do do anything
IpDetector();
// Get the first potential IP address
// Set value of input to remainder of string.
string GetPotentialIpString(string& input);
bool extractIPv4(const string& str, unsigned long& outAddress, int& outPort);
private:
bool IsValidCharacter(char c);
};

```

**** end of IpDetector.hpp ****

Based on the content above, please provide a response to the user query.

User query: Generate a function definition for extractIPv4.

Inputs:

str - string to be parsed

outAddress - Address of found IP address as a single integer. If no IP address is found, this should be 0.

outPort - Address of found port. If no port is present, this should be -1.

This should return a boolean stating whether an IP address was found.

Notes:

- Use GetPotentialIpString to find a string that could be an IP address

- Attempt to construct a `IpAddress` object using that string. If successful, pull the integer value and port from it to their respective variables and return true.
- If an exception is thrown in the `IpAddress` constructor, repeat until either a `IpAddress` is successfully constructed or the input string is empty.
- If the input string is empty, no IP address exists.

Assistant

Here's a straightforward implementation that follows the exact behaviour you described:

```
// IpDetector.cpp (add after existing includes)

#include <stdexcept>          // for std::exception
#include "IpAddress.hpp"      // to construct IpAddress

bool IpDetector::extractIPv4(const std::string& str,
                             unsigned long& outAddress,
                             int& outPort)
{
    /* -----
     * 1. Initialise outputs - if nothing is found we return false and
     *    leave the arguments unchanged (they'll be set to 0 / -1 in the
     *    calling code if needed).
     * ----- */
    outAddress = 0;
    outPort    = -1;

    /* -----
     * 2. Work on a mutable copy of the input - GetPotentialIpString()
     *    consumes characters from the front, so we can keep looping
     *    until the string is exhausted or a valid IP is found.
     * ----- */
    std::string remaining = str;

    /* -----
     * 3. Keep pulling potential-IP substrings until we either succeed
     *    or run out of data.
     * ----- */
    while (!remaining.empty())
    {
        // Grab the next chunk that could be an IP address
        std::string candidate = GetPotentialIpString(remaining);

        /* If nothing was extracted (e.g. leading non-valid char) we break -
         * the loop will exit because remaining has been advanced to the
         * first invalid character, which is still part of 'remaining'.
         */
        if (candidate.empty())
            break;

        try
        {
```

```

        // Try to parse it with IPAddress - this may throw
        IPAddress ip(candidate);

        /* -----
         * 4. If we get here the constructor succeeded - pull the
         *    integer representation and port out.
         * ----- */
        outAddress = static_cast<unsigned long>(ip);
        outPort    = ip.IsPortDefined() ? ip.GetPort() : -1;
        return true;           // success!
    }
    catch (const std::exception&)
    {
        /* The candidate was not a valid IPv4 address - continue
         * searching with the remainder of the string.
         */
        continue;
    }
}

/* -----
 * 5. No valid IP found - return false and leave outputs at their
 *    default (0 / -1).
 * ----- */
return false;
}

```

What this does

Step	Action
Init	<code>outAddress = 0; outPort = -1;</code>
Loop	While the remaining string isn't empty:
Extra ct	<code>GetPotentialIpString(remaining)</code> gives the next block of digits, dots and possibly a colon. The original string is trimmed to the rest after that block.
Parse	Try constructing an <code>IPAddress</code> . If it throws (invalid IP), catch and continue with the next block.
Succe ss	On success, cast the <code>IPAddress</code> to <code>unsigned long</code> for the 32-bit integer value, fetch the port if defined, and return <code>true</code> .
Fail	If we exit the loop without a successful parse, return <code>false</code> .

The function strictly follows your specification:

- It repeatedly calls `GetPotentialIpString`.
- It uses the `IPAddress` constructor to validate.
- It sets `outAddress` and `outPort` only when an address is successfully parsed.
- It returns `true` on success, otherwise `false`.